

Terraform 기본

inky4832@daum.net

1. Terraform 개요

- Infrastructure as Code (IaC)의 개념 이해
- Terraform 기본 이해
- Terraform과 인프라 관리 이해

1.1 IaC 개요

개념

- 인프라는 어플리케이션을 실행하기 위해 필요한 모든 IT 자원을 의미함.
- IaC 는 인프라를 사람이 생성하지 않고 코드로 정의하여 자동으로 생성하는 기술임.

인프라(Infra)

- Virtual Machine(EC2)
- Network(VPC, Subnet)
- Security Group
- Kubernetes Cluster
- Load Balancer
- Database
- Storage(S3)
-

1.1 IaC 개요

등장 배경

- 동일한 환경을 반복 생성하기 어렵다.
- 사람의 실수(Human Error)가 발생할 수 있다.
- 변경 이력 추적이 어렵다.
- 협업 환경에서 일관성을 유지하기 어렵다.
- 운영 환경과 개발 환경이 서로 달라질 가능성이 높다.

장점

- 인프라 구축 및 변경 자동화
- 구성 일관성 유지
- 비용 관리
- 버전 관리
- CI/CD 연계
- 인프라 테스트
- 보안 자동화

2.1 Terraform 개요

개념

- HashiCorp 에서 개발한 Infrastructure as Code (IaC) 도구.

특징

- 코드를 이용하여 인프라를 생성
- 여러 Cloud Provider 지원
- 선언형 (Declarative) 방식 지원
- 동일한 환경을 반복해서 생성 가능
- Git 과 함께 버전 관리 가능
- CI/CD 와 쉽게 연동 가능

2.1 Terraform 개요

선언형 구성 (Declarative Configuration)

- 원하는 최종 상태 (Desired State)를 정의하면, 해당 상태를 만들기 위한 과정은 Terraform이 자동으로 수행됨.
- 최종적으로 원하는 상태를 정의하는 방식이며, 동일한 코드를 여러 번 실행해도 항상 같은 결과를 얻는 멍등성 특징을 가짐.

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}
```

선언형 구성 장점

- 코드가 간결하다.
- 유지보수가 쉽다.
- 변경 사항을 쉽게 확인할 수 있다.
- 동일한 환경을 반복 생성할 수 있다.
- 사람의 실수를 줄일 수 있다.

2.1 Terraform 개요

예시 1

AWS Console 방식

- 1) S3
- 2) Create Bucket
- 3) Bucket 이름 입력
- 4) Create

Terraform 방식

```
resource "aws_s3_bucket" "example" {  
  bucket = "terraform-demo-bucket"  
}
```

실행

```
terraform apply
```

3.1 Terraform CLI

개념

- Terraform을 실행하기 위한 명령행 프로그램.
- Terraform은 별도의 GUI 없이 CLI를 중심으로 동작하며, 사용자가 작성한 Terraform 구성파일(*.tf)을 읽어 인프라를 생성하거나 변경함.
- Terraform CLI는 Go 언어로 개발, 다양한 운영체제(Window, Linux, macOS) 지원.
- <https://developer.hashicorp.com/terraform/install>

The screenshot shows the 'Install Terraform' page. At the top, there's a purple HashiCorp logo and the text 'Install Terraform' with a dropdown menu showing '1.15.8 (latest)'. Below this, there are two main sections: 'macOS' and 'Windows'. The 'macOS' section has a 'Package manager' subsection with a code block containing 'brew tap hashicorp/tap' and 'brew install hashicorp/tap/terraform', and a 'Binary download' subsection with two options: 'AMD64' and 'ARM64', both with 'Download' links. The 'Windows' section is highlighted with a red border and has a 'Binary download' subsection with two options: '386' and 'AMD64', both with 'Download' links. The '386' option is highlighted with a red border.

macOS

Package manager

```
brew tap hashicorp/tap  
brew install hashicorp/tap/terraform
```

Binary download

Architecture	Version	Download
AMD64	Version: 1.15.8	Download
ARM64	Version: 1.15.8	Download

Windows

Binary download

Architecture	Version	Download
386	Version: 1.15.8	Download
AMD64	Version: 1.15.8	Download

3.1 Terraform CLI

동작 과정

- 1) Terraform Code 작성
- 2) terraform init
- 3) terraform validate
- 4) terraform fmt
- 5) terraform plan
- 6) terraform apply
- 7) AWS Infrastructure 생성

3.1 Terraform CLI

주요 Terraform 명령어

명령어	설명
terraform init	Terraform 프로젝트를 초기화
terraform validate	Terraform 코드의 문법을 검사
terraform fmt	Terraform 코드를 표준 형식으로 자동 정렬
terraform plan	Terraform 코드와 현재 Infrastructure 를 비교하여 변경 사항을 미리 확인
terraform apply	Terraform 계획을 실제 AWS 환경에 적용
terraform destroy	Terraform이 관리하는 모든 리소스를 삭제

4.1 Terraform Provider

개념

- Terraform과 클라우드 서비스 사이를 연결하는 플러그인(Plugin).
- Terraform 자체는 AWS나 Azure의 API를 직접 호출하지 않고, 대신 Provider를 통해서 각 서비스의 API와 통신하여 리소스를 생성하거나 수정.
- Provider는 Terraform이 다양한 플랫폼을 동일한 방식으로 사용할 수 있도록 해주는 핵심 구성 요소임.

주요 역할

- 클라우드 API 호출
- 리소스 생성 및 삭제
- 리소스 속성 조회
- Terraform 코드와 클라우드 서비스 간의 데이터 변환

4.1 Terraform Provider

대표적인 Provider

구분	Provider 예시
Cloud	AWS, Azure, Google Cloud
Container	Kubernetes, Docker
Version Control	GitHub, GitLab
Monitoring	Grafana, Datadog
Database	MySQL, PostgreSQL
DNS	Cloudflare, CloudFront

4.1 Terraform Provider

AWS Provider 설정

- Terraform에서 AWS를 사용하려면 Provider를 먼저 선언해야 함.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "ap-northeast-2"  
}
```

4.1 Terraform Provider

Provider 다운로드

- Provider는 프로젝트 생성 시 자동으로 포함되는 것이 아님.
- terraform init 명령어를 실행하면 필요한 Provider가 다운로드됨.

주요 특징

- Provider를 변경하거나 새로운 Provider를 추가한 경우에도 다시 terraform init 명령어를 실행해야 됨.
- Provider는 지속적으로 새로운 기능이 추가되고 기존 기능이 변경됨.

버전 표현식	의미
= 5.0.0	정확하게 5.0.0 만 사용
>= 5.0.0	5.0.0 이상
<= 5.0.0	5.0.0 이하
~> 5.0.0	5.X 버전 사용
!= 5.0.0	5.0.0 제외

5.1 Resources 개요

개념

- Terraform에서 Resource는 생성하거나 관리할 대상(Resource)을 의미함.
- 각각을 하나의 Resource Block으로 정의하여 관리함.

Resource 예

- VPC
- Subnet
- Internet Gateway
- Route Table
- Security Group
- EC2
- ALB
- RDS
- S3
- IAM
- ..

5.2 Resources Block 구조

기본 구조

```
resource "<RESOURCE_TYPE>" "<RESOURCE_NAME>" {  
    property = value  
}
```

AWS EC2 인스턴스 예

```
resource "aws_instance" "web" {  
    ami = "ami-0e123456789abcd12"  
    instance_type = "t3.micro"  
}
```


5.2 Resources Block 구조

Resource Type 예

Resource Type	생성되는 AWS 리소스
<code>aws_vpc</code>	VPC
<code>aws_subnet</code>	Subnet
<code>aws_instance</code>	EC2
<code>aws_security_group</code>	Security Group
<code>aws_lb</code>	Application Load Balancer
<code>aws_db_instance</code>	Amazon RDS
<code>aws_s3_bucket</code>	Amazon S3

5.3 Resources Dependency

개념

- Terraform의 장점 중 하나는 Resource 간의 의존성을 자동으로 분석함.
- 즉, Resource 생성 순서를 자동으로 결정 가능.
- `depends_on` 을 사용하여 명시적으로 의존성을 지정할 수도 있음.

```
resource "aws_subnet" "public" {  
  ...  
}  
  
resource "aws_instance" "web" {  
  subnet_id = aws_subnet.public.id  
}
```

```
resource "aws_instance" "web" {  
  ...  
  
  depends_on = [  
    aws_internet_gateway.main  
  ]  
}
```

5.4 Resources 참조(Reference)

개념

- Terraform에서는 다른 Resource의 값을 쉽게 참조할 수 있음.
- 형식 : **RESOURCE_TYPE.RESOURCE_NAME.PROPERTY**

```
resource "aws_subnet" "public" {  
    ...  
}  
  
# 참조  
aws_subnet.public.id  
  
resource "aws_instance" "web" {  
    subnet_id = aws_subnet.public.id  
}  
  
# 참조  
aws_instance.web.subnet_id
```

5.5 Resources 와 Data Source

개념

- Terraform에서는 AWS 리소스를 생성할 수도 있고, 이미 존재하는 리소스의 정보를 조회하여 사용할 수도 있음.
- 이를 위해 Terraform은 Resource와 Data Source라는 두 가지 구성을 제공함.

Resource

- 새로운 인프라를 생성(Create), 수정(Update), 삭제>Delete)하는 구성 요소임.
- Terraform이 관리하는 대상이며, 실행 결과는 Terraform State 파일에 저장됨.

Data Source

- 이미 존재하는 리소스의 정보를 조회(Read)하는 구성요소임.
- Terraform은 해당 리소스를 생성하거나 삭제하지 않고, 필요한 속성(Attribute)만 참조하여 다른 Resource에서 사용할 수 있도록 제공.

5.5 Resources 와 Data Source

샘플 예

Data Source

```
data "aws_vpc" "default" {  
  default = true  
}
```

Resource

```
resource "aws_security_group" "web" {  
  name = "web-sg"  
  vpc_id = data.aws_vpc.default.id  
}
```

5.5 Resources 와 Data Source

Data Source 예

Data Source	활용 목적
aws_ami	최신 Amazon Linux, Ubuntu AMI 조회
aws_vpc	기존 VPC 정보 조회
aws_subnet	기존 Subnet 정보 조회
aws_security_group	기존 Security Group 조회
aws_route53_zone	Hosted Zone 조회
aws_acm_certificate	ACM 인증서 조회
aws_iam_role	기존 IAM Role 조회
aws_availability_zones	사용 가능한 AZ 목록 조회

6.1 Variables 개요

개념

- Terraform에서는 동일한 코드를 다양한 환경에서 재사용할 수 있도록 Variable(변수) 기능을 제공함.

Variable 선언 및 사용

```
variable "instance_type" {  
    description = "EC2 Instance Type"  
    type        = string  
    default     = "t3.micro"  
}
```

변수 사용

```
resource "aws_instance" "web" {  
    ami = data.aws_ami.amazon_linux.id  
    instance_type = var.instance_type  
}
```

6.1 Variables 개요

변수 타입

- Terraform에서는 다양한 데이터 타입을 지원함.

타입	예제
string	"t3.micro"
number	2
bool	true
list	["a","b"]
map	{ env="dev" }
object	{ cpu:2, memory:4 }

6.1 Variables 개요

terraform.tfvars

- 실무에서는 변수 값을 외부 파일로 관리하는 것이 일반적임.
- 실행 시 자동으로 읽음.

```
instance_type = "t3.small"  
region = "ap-northeast-2"
```

7.1 Outputs 개요

개념

- Terraform은 리소스를 생성한 후 중요한 값을 출력(Output)할 수 있음.
- 예를 들어 생성된 EC2의 Public IP, ALB DNS 이름, VPC ID 등을 출력하여 다른 모듈이나 사용자에게 전달할 수 있음.
- 즉, Output은 생성된 리소스의 정보를 반환하거나 다른 모듈에서 사용할 수 있도록 제공하는 기능임.

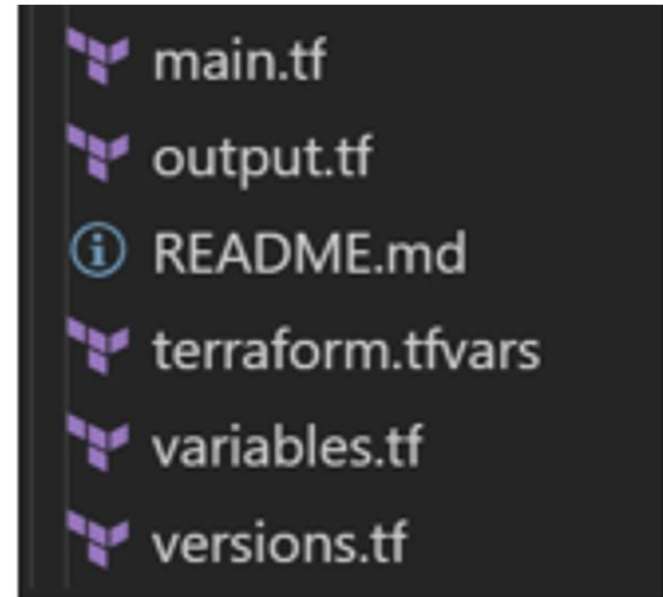
```
output "public_ip" {  
  value = aws_instance.web.public_ip  
}
```

8.1 표준 프로젝트 구조

개념

- Terraform은 파일 이름에 대한 강제 규칙은 없음.
- 커뮤니티에서는 표준적인 프로젝트 구조를 권장함.

```
terraform-project/  
  main.tf  
  provider.tf  
  variables.tf  
  output.tf  
  versions.tf  
  terraform.tfvars  
  .terraform.lock.hcl  
  README.md
```



8.2 Terraform Lock File

개념

- terraform init 명령어를 실행하면 .terraform.lock.hcl 파일이 생성됨.
- 이 파일은 Provider의 버전과 해시 정보를 저장하고, 팀원 간 동일한 실행환경을 유지하기 위해 Git 저장소에 포함시킴.

역할

- Provider 버전 고정
- Provider 무결성(Hash) 검증
- 팀원 간 동일한 환경 유지

9.1 Terraform State

개념

- Terraform이 관리하는 리소스의 정보를 저장하는 데이터베이스와 같은 역할 담당.

저장 정보 예

- 생성된 리소스 ID
- 리소스 이름
- Public IP
- Private IP
- ARN
- Security Group ID
- Subnet ID
- Route Table ID
- Resource 간의 참조 관계

9.1 Terraform State

동작 과정

Terraform Code (*.tf)

terraform plan

terraform.tfstate

AWS 실제 리소스

차이점(Diff) 계산

terraform apply

9.2 Local State와 Remote State

개념

- Terraform은 State를 저장하는 방법이 두 가지임.

Local State

- 기본 방식
- 파일 형식: terraform.tfstate
- 프로젝트 폴더에 저장됨.

Remote State

- 실무에서 사용하는 방식
- S3 + DynamoDB 주로 사용됨

10.1 Variable 심화

개념

- Terraform에서는 동일한 코드를 여러 환경(Development, Production)에서 재사용하기 위해 변수(Variable) 기능을 제공함.
- 즉, 중앙에서 관리되는 재사용 가능한 값이며, Input Variables, Local Variables, Outputs로 구분됨

장점

- 코드 재사용
- 환경별 설정 분리
- 유지보수 용이
- 협업 편의성
- 코드 가독성 향상

활용

- Region
- EC2 Instance Type
- VPC CIDR
- Subnet CIDR
- 프로젝트 이름
- 환경(dev, prod)

10.1 Variable 심화

사용 예

```
resource "aws_instance" "web" {  
  
    instance_type = "t3.micro"  
  
}
```

```
variable "instance_type" {  
  
    default = "t3.micro"  
  
}  
  
resource "aws_instance" "web" {  
  
    instance_type = var.instance_type  
  
}
```

10.1 Variable 심화

Variables 종류

종류	역할
Input Variable	외부에서 값을 입력 받는 변수
Local Variable	코드 내부에서 계산하여 사용하는 변수
Output Value	생성 결과를 출력하는 변수

Variables 사용 흐름

사용자 입력

Input Variable

Terraform Code

Local Variable 계산

AWS 리소스 생성

Output 출력

10.2 Input Variable

개념

- Terraform 실행 시 외부에서 값을 전달받기 위한 변수임.
- 프로그램 언어에서 함수(Function)의 파라미터(Parameter)와 같은 역할을 하며, 동일한 Terraform 코드를 다양한 환경에서 재사용할 수 있도록 지원.

Input Variable 선언

- 선언은 `variable` 블록을 사용하여 선언함.
- 선언한 후에는 `var.변수명` 형식으로 사용함.

```
variable "instance_type" {  
  description = "EC2 Instance Type"  
  
  type      = string  
  
  default   = "t3.micro"  
}
```

10.2 Input Variable

Optional Arguments

- Variable에는 다양한 옵션을 지정할 수 있음.

옵션	설명
default	기본값
type	데이터 타입
description	변수 설명
validation	입력값 검증
sensitive	민감한 정보 보호

10.2 Input Variable

Variable Data Type

- Terraform은 여러 가지 데이터 타입을 지원함.

타입	설명	예시
string	문자열	"t3.micro"
number	숫자	2
bool	논리값	true
list	목록	["a","b"]
map	key-value	{ env="dev" }
object	여러 속성	{ cpu=2, memory=4 }

10.2 Input Variable

String

- 가장 많이 사용하는 타입.
- 사용 예
 - Region
 - Instance Type
 - Project Name

```
variable "instance_type" {  
  type = string  
  default = "t3.micro"  
}
```

10.2 Input Variable

Number

- 숫자형 변수 타입.
- 사용 예
 - EC2 개수
 - EBS 크기
 - 포트 번호

```
variable "instance_count" {  
    type = number  
    default = 2  
}
```

10.2 Input Variable

Boolean

- 참(true) 또는 거짓(false) 변수 타입.
- 사용 예
 - CloudWatch 사용 여부
 - Public IP 할당 여부

```
variable "enable_monitoring" {  
  type = bool  
  default = true  
}
```


10.2 Input Variable

List

- 여러 개의 값을 순서대로 지정하는 변수 타입.

```
variable "availability_zones" {  
  type = list(string)  
  default = [  
    "ap-northeast-2a",  
    "ap-northeast-2c"  
  ]  
}
```

참조

```
var.availability_zones[0]
```

10.2 Input Variable

Map

- Key와 Value 형태의 데이터를 저장한다.

```
variable "common_tags" {  
  type = map(string)  
  default = {  
    Environment = "dev"  
    Team = "Cloud"  
  }  
}
```

참조

```
var.common_tags["Environment"]
```

10.2 Input Variable

Object

- 여러 속성을 하나의 변수로 관리한다.

```
variable "server" {  
  type = object({  
    instance_type = string  
    volume_size  = number  
  })  
}
```

참조

```
instance_type = var.server.instance_type
```

10.3 Variables 값 전달 방법

개념

- Terraform에서는 Variable을 선언하는 것만으로는 값을 사용할 수 없음.
- 실행 시점에 Variable에 실제 값을 전달해야 하며, 이를 위해서 다양한 방법이 제공됨.
- 동일한 Variable이라고 값을 전달하는 방법은 다양하며, Terraform은 우선순위에 따라 최종 값을 결정한다.

전달방법 5가지 (우선순위)

- 1) Command Line
- 2) terraform.tfvars
- 3) *.auto.tfvars
- 4) Environment Variables
- 5) Default Value

10.3 Variables 값 전달 방법

Command Line

- 실행 시 `-var` 옵션을 사용.
- 여러 개 지정도 가능

```
terraform apply -var="instance_type=t3.micro"
```

```
terraform apply \  
-var="instance_type=t3.large" \  
-var="region=ap-northeast-2"
```

10.3 Variables 값 전달 방법

terraform.tfvars

- 가장 많이 사용하는 방법
- 프로젝트 루트에 terraform.tfvars 파일을 생성

```
# terraform.tfvars  
region = "ap-northeast-2"  
instance_type = "t3.micro"  
environment = "dev"
```

10.3 Variables 값 전달 방법

사용자 정의 tfvars

- 환경별 설정을 관리하기 위해 여러 개의 tfvars 파일 사용 가능.

```
# dev.tfvars  
  
instance_type = "t3.micro"  
  
# prod.tfvars  
  
instance_type = "t3.large"
```

```
# 실행  
  
terraform apply \  
-var-file="prod.tfvars"
```

10.3 Variables 값 전달 방법

auto.tfvars

- Terraform 실행 시 *.auto.tfvars 파일을 자동으로 읽음
- 공통 설정을 관리할 때 주로 사용.

common.auto.tfvars

network.auto.tfvars

10.3 Variables 값 전달 방법

Environment Variables

- 운영 환경에서는 환경 변수도 자주 사용됨
- 형식: TF_VAR_변수명

Linux 환경

```
export TF_VAR_region="ap-northeast-2"
```

```
export TF_VAR_instance_type="t3.medium"
```

Window 환경 (PowerShell)

```
$env:TF_VAR_region="ap-northeast-2"
```

10.3 Variables 값 전달 방법

Default Value

- Variable에서 지정한 기본값.
- 다른 값이 전달되지 않았을 때 자동으로 사용됨.

```
variable "region" {  
    default = "ap-northeast-2"  
}
```

10.4 Terraform Workspace

개념

- Terraform에서는 개발, 운영과 같이 여러 환경을 관리해야 하는 경우가 많음.
- tfvars 파일 방법 및 Workspace 방법이 있음.

tfvars 방법

- 이 방법은 Variable 값만 변경하는 기능임.
- 즉 Terraform 코드는 그대로 두고 환경별 변수만 변경하여 실행함.

```
# main.tf 는 동일함
# dev.tfvars
    instance_type = "t3.micro"
# prod.tfvars
    instance_type = "t3.large"
```

```
terraform apply \
-var-file="dev.tfvars"

terraform apply \
-var-file="prod.tfvars"
```

10.4 Terraform Workspace

Workspace

- Terraform State를 분리하는 기능임
- 즉, 개발환경과 운영환경이 서로 다른 State 파일을 사용하도록 만드는 기능임.
- State가 다르기 때문에 두 환경이 서로 영향을 주지 않음.

main.tf



Workspace(dev)



terraform.tfstate(dev)

main.tf



Workspace(prod)



terraform.tfstate(prod)

10.4 Terraform Workspace

Workspace 명령어

명령어	설명
terraform workspace show	현재 Workspace 확인
terraform workspace new dev	새로운 Workspace 생성
terraform workspace list	목록 조회
terraform workspace select prod	Workspace 변경
terraform workspace delete dev	Workspace 삭제

10.5 Input Variable Validation

개념

- Variable에 아무값이나 입력되는 것을 방지하기 위해서 Validation 기능을 제공.
- Validation을 사용하면 사용자가 잘못된 값을 입력했을 때, Terraform 실행 전에 오류를 발생시켜 잘못된 인프라 생성과 배포를 방지할 수 있음.

```
variable "instance_type" {  
  type = string  
  validation {  
    condition = contains(  
      ["t3.micro", "t3.small", "t3.medium"],  
      var.instance_type  
    )  
    error_message = "지원하지 않는 Instance Type 입니다."  
  }  
}
```

10.6 Local Variable

개념

- Local Variable은 Terraform 내부에서 계산하여 사용하는 변수임.
- 사용자가 값을 입력하지 않으며, 중복되는 값을 줄이고 코드의 가독성을 높이는데 사용됨.

```
locals {  
  project_name = "banking"  
  environment = "dev"  
}
```

사용

```
local.project_name
```

10.6 Local Variable

사용 예 1

```
# 문자열 조합
locals {
  instance_name =
    "${var.project}-${var.environment}-web"
}
# 결과
banking-dev-web
```


10.6 Local Variable

사용 예 2

공통 Tag 관리

locals {

common_tags = {

Project = var.project

Environment = var.environment

ManagedBy = "Terraform"

}

}

사용

tags = local.common_tags

10.6 Local Variable

Local 과 Input Variable 차이

Input Variable	Local Variable
외부에서 입력	내부에서 계산
사용자 변경 가능	변경 불가
<code>var.xxx</code>	<code>local.xxx</code>

10.7 Output Values

개념

- Terraform은 리소스를 생성한 후 생성 결과를 확인할 수 있도록 Output Value 기능을 제공함.
- Output은 Terraform 실행이 완료된 후 필요한 정보를 화면에 출력하거나, 다른 Module 또는 시스템에서 재사용할 수 있도록 지원하는 기능임.

활용

EC2 생성



Public IP 출력

ALB 생성



DNS Name 출력

10.7 Output Values

기본구조

```
output "public_ip" {  
  description = "EC2 Public IP"  
  value = aws_instance.web.public_ip  
  sensitive = false  
}
```

실행결과

Outputs:

```
public_ip = "43.xxx.xxx.xxx"
```

10.7 Output Values

terraform output 명령어

- Terraform은 Output 만 따로 조회 가능.

전체 조회

```
terraform output
```

특정 Output 조회

```
terraform output public_ip
```

JSON 형태

```
terraform output -json
```

10.8 보간법 (Interpolation)

개념

- Terraform에서는 문자열 안에 변수나 Resource 값을 삽입하는 기능 지원.

기본문법

```
locals {  
  instance_name = "${var.project}-web"  
}
```

10.9 Conditions (조건식)

개념

- Terraform에서는 조건에 따라 다른 값을 선택하거나, 특정 Resource를 생성 또는 생성하지 않도록 설정 가능. (중첩 가능)
- 반드시 지정된 양쪽 value값이 동일한 타입이어야 됨.

기본문법

```
condition ? true_value : false_value
```

```
variable "environment" {  
  default = "dev"  
}
```

```
locals {  
  instance_type =  
    var.environment == "prod" ?  
      "t3.large" :  
      "t3.micro"  
}
```

10.10 Iteration (반복문)

개념

- Terraform에서는 여러 개의 동일한 Resource를 생성할 때 반복문을 사용함.

대표적인 반복 기능 3 가지

count

for_each

for expression

10.10 Iteration (반복문)

count 반복

- 가장 단순한 반복문임
- `count.index` 사용하여 인덱스 참조 가능

```
resource "aws_instance" "web" {  
  count = 3  
}
```

결과

`web[0]`

`web[1]`

`web[2]`

10.10 Iteration (반복문)

for_each 반복

- Map 또는 Set을 반복
- each.key, each.value 사용

```
resource "aws_instance" "web" {  
  for_each = toset([  
    "frontend",  
    "backend",  
    "admin"  
  ])  
  tags = {  
    Name = each.key  
  }  
}
```

결과

```
frontend  
backend  
admin
```

10.10 Iteration (반복문)

for expression 반복

- 데이터를 변환하는 목적임.

```
locals {  
  names = ["web","db","api"]  
  upper_names = [  
    for name in local.names :  
    upper(name)  
  ]  
}
```

결과

WEB

DB

API

10.11 Built-in Functions

개요

- Terraform은 변수와 데이터를 쉽게 처리할 수 있도록 다양한 Built-in 함수(내장 함수)를 제공함.
- 내장함수를 사용하면 문자열 처리, 리스트 변환, Map 검색, 기본값 설정, 데이터 병합 등을 간단하게 수행할 수 있음.

함수	기능	샘플
length()	데이터의 개수를 반환	length("Terraform") length(var.subnets)
upper()	문자를 대문자로 변환	upper("terraform")
lower()	문자를 소문자로 변환	bucket = lower(var.project_name)
replace()	문자를 변경	replace("terraform", "t", "T")
split	문자를 나눔	split(",", "web,db,redis")
join()	리스트를 문자열로 변환	join(",", ["web", "db", "redis"])
contains()	리스트안에 값이 있는지 확인	contains(["dev","prod"], "dev")

10.11 Built-in Functions

함수	기능	샘플
<code>lookup()</code>	Map에서 값을 찾음	<code>lookup(var.instance_types, "prod","t3.small")</code>
<code>merge()</code>	Map을 합침	<code>merge(local.common_tags, local.project_tags)</code>
<code>concat()</code>	List를 합침	<code>concat(["a","b"], ["c","d"])</code>
<code>flatten()</code>	중첩된 List 평탄화	<code>flatten([["a","b"],["c","d"]])</code>
<code>try()</code>	오류 발생 시 기본값 사용	<code>try(var.name, "default")</code>
<code>coalesce()</code>	첫 번째 null 이 아닌값 반환	<code>coalesce(null, null, "Terraform")</code>

10.12 Dynamic Block

개요

- Terraform에서는 리소스 내부의 중첩 블록(Nested Block)을 반복해서 생성해야 하는 경우가 많음.
- ex. 보안그룹에 여러 개의 인바운드 규칙 설정
- **dynamic** 블록을 사용하면 이러한 중첩 블록을 반복문처럼 자동 생성할 수 있음.

```
variable "ingress_ports" {  
  default = [  
    22,  
    80,  
    443  
  ]  
}
```

```
dynamic "ingress" {  
  for_each = var.ingress_ports  
  content {  
    from_port = ingress.value  
    to_port   = ingress.value  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

10.12 Dynamic Block

주요 사용 예

리소스	활용 예
Security Group	Ingress/Egress 규칙 반복 생성
ALB Listener	Listener Rule 반복 생성
IAM Policy	Statement 반복 생성
Route 53	여러 레코드 생성
Kubernetes	반복되는 설정 블록 생성

10.12 Dynamic Block

Dynamic Block vs for_each

항목	for_each	dynamic
반복 대상	리소스 자체	리소스 내부 블록
생성 결과	여러 개의 리소스	하나의 리소스 안에 여러 블록
대표 사용 예	EC2 여러 대 생성	Security Group Ingress 여러 개 생성

11.1 Advanced Resources

개요

- Terraform에서는 단순히 리소스를 생성하는 것뿐만 아니라, 기존 인프라를 조회하고 (Data Sources), 리소스 간의 실행 순서를 제어하며(Dependencies), 반복되는 블록을 동적으로 생성(Dynamic Blocks)하고, 리소스의 생성 및 삭제 방식(Lifecycle)을 제어할 수 있음.
- 이러한 기능들은 중/대규모 프로젝트에서 Terraform을 효율적으로 운영하기 위해서 반드시 이해해야 하는 핵심 기능임.

11.2 Data Source

개요

- Terraform은 일반적으로 새로운 인프라를 생성하는 데 사용되지만, 이미 존재하는 AWS 리소스를 조회하여 사용할 수도 있음.
- 이때 사용하는 기능이 Data Source 임.
- 즉 Data Source는 리소스를 생성하지 않고, 기존 리소스의 정보를 조회(Query)하여 Terraform 코드에서 사용할 수 있도록 함.

대표 예

- 이미 생성된 VPC
- 기존 Subnet
- 최신 Amazon Linux AMI
- Route 53 Hosted Zone
- IAM Role
- ...

11.2 Data Source

기본 문법

- Data Source는 data 블록으로 선언함.

```
data "aws_vpc" "main" {
  tags = {
    Name = "production-vpc"
  }
}

data.aws_vpc.main.id
```

항목	설명
data	Data Source 선언
aws_vpc	조회할 리소스 종류
main	Data Source 이름

11.2 Data Source

샘플 예 1

- 최신 Amazon Linux 조회해서 EC2 생성

```
data "aws_ami" "amazon_linux" {  
  
    most_recent = true  
    owners = ["amazon"]  
    filter {  
        name = "name"  
        values = ["al2023-ami-*"]  
    }  
}
```

```
resource "aws_instance" "web" {  
  
    ami = data.aws_ami.amazon_linux.id  
    instance_type = "t3.micro"  
}
```

11.2 Data Source

샘플 예 2

- 기존 VPC 조회해서 EC2 생성

```
data "aws_vpc" "main" {  
  
  tags = {  
    Name = "production-vpc"  
  }  
}
```

```
resource "aws_instance" "web" {  
  
  subnet_id = data.aws_subnet.public.id  
  
}
```

11.2 Data Source

Data Source 예

Data Source	활용 목적
aws_ami	최신 Amazon Linux, Ubuntu AMI 조회
aws_vpc	기존 VPC 정보 조회
aws_subnet	기존 Subnet 정보 조회
aws_security_group	기존 Security Group 조회
aws_route53_zone	Hosted Zone 조회
aws_acm_certificate	ACM 인증서 조회
aws_iam_role	기존 IAM Role 조회
aws_availability_zones	사용 가능한 AZ 목록 조회

11.3 Explicit Dependencies

개념

- Terraform은 여러 개의 리소스를 생성할 때, 리소스 간의 의존성(Dependency)을 분석하여 적절한 순서로 생성함.
- 대부분의 경우 Terraform은 코드만 보고 자동으로 의존성을 계산하지만, 코드만으로는 알 수 없는 관계도 존재함.
- 이러한 경우 `depends_on` 메타인수를 사용하여 명시적으로 의존성을 설정함.

Terraform Dependency Graph

- Terraform은 실행 전에 모든 Resource를 분석하여 Dependency Graph를 생성함.

Terraform Code



Dependency Graph 생성 및 생성 순서 계산



`terraform apply`



AWS Resource 생성

11.3 Explicit Dependencies

기본 문법

```
resource "aws_instance" "web" {  
    ami = data.aws_ami.amazon_linux.id  
    instance_type = "t3.micro"  
  
    depends_on = [  
        aws_iam_role_policy.ec2  
    ]  
}
```


11.4 Dynamic Blocks

개념

- Terraform은 Resource 내부에 동일한 형태의 Nested Block(중첩 블록)을 여러 번 작성해야 하는 경우가 많음.
- 이때 리소스 내부의 반복되는 블록을 자동 생성하는 기능임.

```
resource "aws_security_group" "web" {  
  ingress {  
    from_port = 22  
    to_port = 22  
  }  
  ingress {  
    from_port = 80  
    to_port = 80  
  }  
  ingress {  
    from_port = 443  
    to_port = 443  
  }  
}
```

```
resource "aws_security_group" "web" {  
  
  dynamic "ingress" {  
    for_each = var.ingress_ports  
    content {  
      from_port = ingress.value  
      to_port = ingress.value  
      protocol = "tcp"  
      cidr_blocks = ["0.0.0.0/0"]  
    }  
  }  
}
```

11.4 Dynamic Blocks

기본 문법

```
dynamic "블록이름" {  
  
    for_each = 반복할 데이터  
  
    # iterator = 변수명 (선택)  
  
    content {  
  
        생성할 내용  
    }  
}
```

```
resource "aws_security_group" "web" {  
  
    dynamic "ingress" {  
        for_each = var.ingress_ports  
        content {  
            from_port = ingress.value  
            to_port   = ingress.value  
            protocol = "tcp"  
            cidr_blocks = ["0.0.0.0/0"]  
        }  
    }  
}
```

11.4 Dynamic Blocks

샘플

```
variable "allowed_ports" {  
  default = [  
    22,  
    80,  
    443  
  ]  
}
```

```
resource "aws_security_group" "web" {  
  name = "web-sg"  
  
  dynamic "ingress" {  
    for_each = var.allowed_ports  
    content {  
      from_port = ingress.value  
      to_port   = ingress.value  
      protocol = "tcp"  
      cidr_blocks = ["0.0.0.0/0"]  
    }  
  }  
}
```

11.5 Resource Lifecycle

개념

- Terraform은 Resource 를 생성(Create), 수정(Update), 삭제(Destroy)하는 전체 과정을 자동으로 관리함. (이것을 Resource Lifecycle 이라고 함)
- 일반적인 경우에는 Terraform이 자동으로 처리하지만, 운영 환경에서는 기본 동작을 변경해야 하는 경우가 있음.
- 이를 위해 Terraform은 Lifecycle Block을 제공함.

기본 문법

```
resource "aws_instance" "web" {  
  
  ...  
  
  lifecycle {  
  
  }  
}
```

11.5 Resource Lifecycle

주요 옵션

주요 옵션	설명
<code>create_before_destroy</code>	새로운 리소스 먼저 생성
<code>prevent_destroy</code>	중요 리소스 삭제 방지
<code>ignore_changes</code>	특정 속성 변경 무시

11.5 Resource Lifecycle

create_before_destroy 옵션

- 기본적으로 Terraform은 기존 리소스 삭제 후 새로운 리소스 생성순으로 동작됨.
- 하지만 서비스가 운영 중이라면 삭제하는 순간 서비스가 중단됨.

기본 문법

```
resource "aws_instance" "web" {  
  ...  
  
  lifecycle {  
  
    create_before_destroy = true  
  
  }  
}
```

11.5 Resource Lifecycle

prevent_destroy 옵션

- 실수로 중요한 Resource를 삭제하는 것을 방지함.

기본 문법

```
resource "aws_db_instance" "prod" {  
  ...  
  
  lifecycle {  
  
    prevent_destroy = true  
  
  }  
}
```

11.5 Resource Lifecycle

ignore_changes 옵션

- Terraform은 실제 AWS와 Terraform State를 비교하여 차이가 있으면 수정하려고 함.
- Terraform 외부에서 변경된 속성을 다시 원래 상태로 되돌리지 않기 위해서 사용.

기본 문법

```
resource "aws_instance" "web" {\n\n  ...\n\n  lifecycle {\n\n    ignore_changes = [\n      tags,\n      desired_capacity\n    ]\n  }\n}
```


11.5 Resource Lifecycle

Lifecycle 옵션 비교

옵션	역할	대표 사용 사례
create_before_destroy	새로운 리소스 먼저 생성	ALB, EC2, Launch Template
prevent_destroy	중요 리소스 삭제 방지	RDS, S3
ignore_changes	특정 속성 변경 무시	Tags, Desired Capacity

12.1 Backends

개념

- Terraform은 Infrastructure 를 코드(IaC)로 관리하기 위해 현재 Infrastructure의 상태(State)를 관리한다.
- 기본적으로 Terraform은 현재 디렉터리에 terraform.tfstate 파일을 생성하여 리소스의 상태를 저장함.
- 혼자 개발하는 경우에는 Local State만으로도 충분하지만, 여러 명이 함께 Terraform을 사용하는 협업 환경에서는 Local State 만으로는 다양한 문제가 발생됨.
- 이를 해결하기 위해서 Terraform은 Backend 기능을 제공함.

주요 기능

- Backend는 Terraform State를 저장할 위치와 저장 방식을 정의하는 기능임.
- AWS 환경에서는 일반적으로 S3를 State 저장소로 사용하고 DynamoDB를 이용하여 State Lock을 구현함.

12.1 Backends

종류

Backend	설명	사용 환경
Local Backend	로컬 PC에 State 저장	개인학습, 테스트
Remote Backend	원격 저장소에 State 저장	협업, 운영 환경

12.2 Local Backend

개념

- Local Backend는 Terraform 의 기본 Backend 임.
- 별도의 설정이 없어도 Terraform은 현재 작업 디렉터리에 State 파일을 생성함.
- 파일명: terraform.tfstate

장점

- 별도 설정이 필요 없음
- 빠르게 실습 가능
- 개인 프로젝트에 적합
- Terraform 학습에 적합

12.2 Local Backend

단점

문제	설명
협업 불가	State 공유 불가능
백업 어려움	PC 장애 시 복구 어려움
Lock 기능 없음	동시 작업 시 충돌 발생
버전 관리 어려움	State 변경 이력 관리 어려움

12.2 Local Backend

주의할 점

- EC2 ID
- RDS Endpoint
- ALB ARN
- IAM Role
- 일부 민감한 정보 등

12.3 Remote Backend

개념

- Remote Backend는 State 파일을 원격 저장소에 저장하는 방식임.

권장 아키텍처

Terraform



Amazon S3



terraform.tfstate



Amazon DynamoDB



State Lock

12.3 Remote Backend

장점

장점	설명
State 공유	여러 개발자가 동일한 State 사용
Lock 지원	동시 작업 방지
버전 관리	S3 Versioning 사용 가능
백업	자동 백업 가능
장애 복구	State 복원 가능

12.4 AWS S3 Backend 구성

개념

- AWS에서 Terraform을 운영할 때 가장 많이 사용하는 Backend 구성임.
- S3는 높은 내구성과 가용성을 제공하며, Versioning 기능을 통해 State 파일의 변경 이력을 관리할 수 있음.
- 또한 DynamoDB와 함께 사용하면 여러 사용자가 동시에 Terraform을 실행하더라도 State 충돌을 방지할 수 있음.

12.4 AWS S3 Backend 구성

S3 Backend 동작 과정

- Terraform이 실행되면 다음과 같은 순서로 동작됨

terraform plan



S3에서 terraform.tfstate 다운로드



Terraform Code 비교



AWS Infrastructure 비교



변경 사항 계산



terraform apply



S3 State 업데이트

12.4 AWS S3 Backend 구성

S3 Bucket 준비

- Backend를 사용하려면 먼저 State를 저장할 Bucket이 있어야 됨

Bucket Name

terraform-state-inky4832

권장 설정

항목	권장
Versioning	활성화
Encryption	활성화 (SSE-S3 또는 SSE-KMS)
Public Access	모든 차단
Lifecycle	필요 시 오래된 버전 정리

12.4 AWS S3 Backend 구성

Backend Block 작성

- Terraform에서는 terraform 블록 내부에 Backend를 선언함.
- Terraform은 이 설정을 이용하여 State를 S3에 저장함.

```
terraform {  
  
  backend "s3" {  
    bucket = "terraform-state-inky4832"  
    key = "prod/network/terraform.tfstate"  
    region = "ap-northeast-2"  
    encrypt = true  
  }  
}
```

12.5 AWS Backend Authentication

개요

- Terraform Backend는 단순히 State를 저장하는 기능만 제공하지 않음.
- Backend가 Amazon S3에 저장되어 있다면 Terraform은 S3 Bucket에 접근하여 State 다운로드, 업로드, 수정, State Lock 확인 등의 작업을 수행해야 됨.
- 따라서 Terraform이 AWS 리소스에 접근할 수 있도록 인증(Authentication)이 반드시 필요함.

Terraform의 인증 구조

Terraform 실행



AWS 자격 증명 확인



Amazon S3 Backend 접근



terraform.tfstate 다운로드



Terraform 실행



State 저장

12.5 AWS Backend Authentication

AWS에서 사용할 수 있는 인증 방식

Terraform은 여러 가지 인증 방식을 지원함.

인증 방식	사용 환경
AWS CLI Credential	개인 PC
Environment Variable	CI/CD
IAM Role	EC2
Github Actions OIDC	Github Actions
IRSA	EKS
Access key 직접 입력	테스트

13.1 AWS Provider와 인증처리

개요

- Terraform은 AWS 리소스를 직접 생성하지 않음.
- AWS Provider를 통해 AWS API를 호출하고, EC2, VPC, RDS, S3와 같은 AWS 리소스를 생성하거나 수정함.
- 따라서 Terraform으로 AWS를 관리하려면 다음 두 가지가 필요함.
 - AWS Provider 설정
 - AWS 인증 정보

13.2 AWS Provider

개요

- Provider는 Terraform과 외부 플랫폼을 연결하는 플러그인임.

terraform apply 실행



AWS Provider가 Resource 설정 해석



AWS EC2 API 호출



EC2 인스턴스 생성



생성 결과를 Terraform State에 저장

13.2 AWS Provider

기본 구성

- Terraform Provider를 사용하려면 terraform 블록과 provider 블록을 작성해야됨.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "ap-northeast-2"  
}
```

13.2 AWS Provider

required_providers

- Terraform 프로젝트에서 어떤 Provider를 사용할 지 선언하는 부분임.
- source는 Provider를 어디에서 다운로드할지 지정함. (제작자/Provider 이름)
- version은 사용할 AWS Provider 버전 범위를 지정함.
- Provider 버전을 지정하지 않으면 프로젝트를 다시 초기화할 때 예상하지 못한 새로운 버전이 설치될 수 있음.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

13.2 AWS Provider

Provider Block

- Provider Block은 AWS Provider의 실제 동작 환경을 설정함.
- 즉, Terraform에서 Resource를 생성하면 모두 서울 리전에 생성됨.

```
provider "aws" {  
  region = "ap-northeast-2"  
}
```

13.2 AWS Provider

Provider 초기화

- Provider 설정을 작성한 후에는 Terraform 초기화를 수행해야 됨
- 명령어: terraform init

Terraform 프로젝트 초기화



required_providers 확인



AWS Provider 다운로드



Provider 버전 확인



.terraform 디렉터리 생성



terraform.lock.hcl 생성

13.3 AWS 인증처리

현재 AWS 계정 확인하기

- Terraform을 실행하기 전에 AWS CLI를 통해 현재 인증 계정을 확인함.
- 실행 결과를 통해서 다음 내용을 확인 가능.
 - 현재 AWS 계정 ID
 - IAM User 또는 IAM Role
 - Terraform이 사용할 인증 주체

```
aws sts get-caller-identity
```

```
# 출력 예시
```

```
{  
  "UserId": "AIDAEXAMPLEUSER",  
  "Account": "123456789012",  
  "Arn": "arn:aws:iam::123456789012:user/terraform-user"  
}
```

13.3 AWS 인증처리

Provider에 Access Key를 직접 작성하는 방식

- 문법적으로는 다음과 같이 Provider Block에 Access Key를 작성할 수 있음.

```
provider "aws" {  
  region    = "ap-northeast-2"  
  access_key = "AKIAEXAMPLE"  
  secret_key = "example-secret-key"  
}
```

문제점

- GitHub에 Credential이 노출될 수 있음.
- 코드 복사 과정에서 Credential이 유출될 수 있음.
- Access Key 교체가 어려움.
- 여러 환경에서 동일한 코드를 재사용하기 어려움.
- 보안 사고 가능성이 커짐.

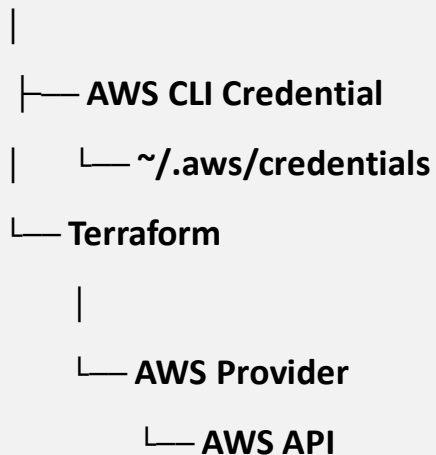
13.4 AWS CLI Authentication

개념

- AWS CLI Authentication은 개인 PC나 관리자용 개발 환경에서 Terraform을 실행할 때 가장 기본적으로 사용하는 인증 방식임.
- 사용자가 aws configure를 실행하면 AWS CLI는 Access Key와 기본 리전 등의 정보를 로컬 Credential 파일에 저장함.
- Terraform Provider는 이 파일을 자동으로 읽어 AWS API를 호출함.

인증 구조

사용자 PC



13.5 GitHub Actions OIDC

개념

- GitHub Actions에서 Terraform을 실행할 때는 AWS Access Key를 GitHub Secrets에 저장하는 방식보다 GitHub OIDC와 AWS IAM Role을 연동하는 방식을 권장함.

동작 구성

GitHub Repository

|
| push 또는 pull request



GitHub Actions Runner

|
| GitHub OIDC Token 발급



AWS IAM OIDC Provider

|
| sts:AssumeRoleWithWebIdentity



AWS IAM Role

|
| 임시 자격 증명 발급



Terraform

|



AWS S3 Bucket 생성

13.5 GitHub Actions OIDC

기존 Access Key 방식과 OIDC 방식 비교

1. 기존 Access Key 방식

IAM User 생성

- |
- ├─ Access Key 발급
- ├─ GitHub Secret에 저장
- ├─ Key Rotation 필요
- └─ 유출 위험 존재

- name: Configure AWS credentials

uses: aws-actions/configure-aws-credentials@v5

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: ap-northeast-2

13.5 GitHub Actions OIDC

기존 Access Key 방식과 OIDC 방식 비교

2. OIDC 방식

GitHub Actions

	- permissions:
— 실행 시 OIDC Token 발급	id-token: write
— AWS IAM Role Assume	contents: read
— 임시 Credential 사용	steps:
— 실행 종료 후 Credential 만료	- name: Configure AWS credentials
	uses: aws-actions/configure-aws-credentials@v5
	with:
	role-to-assume: arn:aws:iam::123456789012:role/cld-github-
	actions-terraform-role
	aws-region: ap-northeast-2

13.5 GitHub Actions OIDC

GitHub OIDC 인증 구성 요소

GitHub Actions OIDC 인증에는 다음 요소가 필요함.

구성 요소	역할
GitHub Repository	Terraform 코드와 Workflow 저장
GitHub Actions Runner	Terraform 명령 실행
GitHub OIDC Provider	GitHub Actions Identity Token 발급
AWS IAM OIDC Provider	GitHub OIDC Token 신뢰
AWS IAM Role	GitHub Actions가 Assume할 Role
Trust Policy	어떤 저장소와 Branch를 신뢰할지 결정
Permission Policy	Terraform이 AWS에서 수행할 작업 결정

13.5 GitHub Actions OIDC

GitHub OIDC의 동작 과정

Workflow가 실행되면 다음 순서로 인증된다.

1. GitHub Actions Workflow 실행
2. permissions.id-token: write 확인
3. GitHub가 Workflow 전용 OIDC JWT 발급
4. configure-aws-credentials Action이 Token을 AWS STS에 전달
5. AWS IAM이 OIDC Provider와 Trust Policy 확인
6. 저장소, Branch, Audience 조건 확인
7. sts:AssumeRoleWithWebIdentity 실행
8. 임시 Access Key, Secret Key, Session Token 발급
9. Terraform AWS Provider가 임시 Credential 사용
10. AWS 리소스 생성

14.1 기존 AWS 인프라와 Terraform 연동

개념

- Terraform은 새로운 인프라를 생성하는 데만 사용하지 않음.
- AWS Management Console, AWS CLI, CloudFormation 또는 수동작업으로 이미 생성된 AWS 리소스도 Terraform 관리 대상으로 전환할 수 있음.

필요 요소

- 기존 인프라를 Terraform으로 관리하기 위해서는 다음 두 요소가 필요함.
 - Terraform 구성 코드
 - 기존 리소스 정보가 등록된 Terraform State

14.2 Greenfield 와 Brownfield

Greenfield

- Greenfield는 기존 리소스가 없는 상태에서 Terraform으로 처음부터 인프라를 구축하는 방식임

Terraform 코드 작성



terraform plan



terraform apply



AWS 리소스 생성



Terraform State 기록

14.2 Greenfield 와 Brownfield

Brownfield

- Brownfield는 이미 AWS에 리소스가 존재하는 상태에서 Terraform 관리를 도입하는 방식임.

AWS Console 또는 AWS CLI로 생성된 기존 리소스



Terraform 코드 작성



Terraform Import



Terraform State에 기존 리소스 등록



terraform plan으로 코드와 실제 환경 비교

14.3 Terraform의 변경계획 생성

개념

- Terraform은 다음 세 가지 정보를 비교하여 변경 계획을 세움.
- 즉, Terraform 코드에 기존 리소스와 동일한 내용을 작성했다고 해서 Terraform이 자동으로 해당 AWS 리소스를 연결하지 않음.

Terraform 구성 코드



Terraform State



실제 AWS 리소스

14.4 기존 인프라를 Terraform으로 관리하는 방법

방법

- 기존 AWS 리소스를 Terraform으로 전환하려면 다음 두 가지 작업이 필요함.

1단계: Terraform 코드 작성

- 기존 리소스에 대응하는 Resource Block을 작성한다.

```
resource "aws_vpc" "existing" {  
  cidr_block = "10.10.0.0/16"  
  
  tags = {  
    Name = "existing-vpc"  
  }  
}
```

14.4 기존 인프라를 Terraform으로 관리하는 방법

2단계: 기존 리소스를 State에 연결

- Terraform Resource 주소와 실제 AWS 리소스 ID를 연결한다.

```
명령어: terraform import  aws_vpc.existing  vpc-0123456789abcdef0  
        terraform plan
```

연결 구조

Terraform Resource 주소

aws_vpc.existing



Terraform State



실제 AWS Resource ID

vpc-0123456789abcdef0

14.4 기존 인프라를 Terraform으로 관리하는 방법

Data Source 와 Import 비교

구분	Data Source	import
목적	기존 리소스 조회	기존 리소스를 Terraform 관리 대상으로 전환
선언 방식	data Block	resource Block
State 저장	조회 결과 저장	실제 리소스와 Resource 주소 매핑
리소스 변경	변경 안됨	변경 가능
리소스 삭제	변경 안됨	코드와 실행 명령에 따라 가능
사용 예	기본 VPC ID 참조	기존 VPC의 설정을 Terraform으로 관리

14.4 기존 인프라를 Terraform으로 관리하는 방법

2단계: 기존 리소스를 State에 연결

- Terraform Resource 주소와 실제 AWS 리소스 ID를 연결한다.

```
명령어: terraform import  aws_vpc.existing  vpc-0123456789abcdef0  
  
terraform plan
```

연결 구조

Terraform Resource 주소

aws_vpc.existing



Terraform State



실제 AWS Resource ID

vpc-0123456789abcdef0

15.1 Module

개념

- Terraform 프로젝트가 작을 때는 하나의 디렉터리에 모든 리소스를 작성해도 큰 문제가 없음. 하지만 프로젝트 규모가 커지면 관리가 어려워짐.
- 이러한 문제를 해결하기 위해 관련된 리소스를 하나의 재사용 가능한 단위로 묶는 기능이 Terraform Module임.

특징

- Module을 함께 사용되는 여러 Resource를 저장하는 컨테이너로 간주.
- 코드 재사용, 관심사 분리, 모듈화 및 DRY 원칙을 적용하기 위한 방법임.
- Root Module이 전체 인프라를 조율하는 Orchestrator 역할을 하게 됨.

15.1 Module

샘플

VPC에서 반드시 필요한 리소스들

Public Subnet

Private Subnet

Internet Gateway

Route Table

Route Table Association

NAT Gateway



modules/

└─ network/

│ └─ main.tf

│ └─ variables.tf

└─ outputs.tf

```
module "network" {  
  source = "../modules/network"  
  vpc_cidr      = "10.0.0.0/16"  
  public_subnet_cidr = "10.0.1.0/24"  
}
```

15.2 Root Module과 Child Module

개념

- Terraform에서는 현재 Terraform 명령을 실행하는 디렉터리를 Root Module이라고 함.
- Root Module에서 호출되는 다른 Module은 Child Module 이라고 함.

기본 구조

```
terraform-project/  
├── main.tf  
├── variables.tf  
├── outputs.tf  
└── modules/  
    ├── network/  
    │   ├── main.tf  
    │   ├── variables.tf  
    │   └── outputs.tf
```

15.2 Root Module과 Child Module

Root Module 역할

- 전체 인프라를 조합하고 실행 환경에 맞는 값을 전달함.

샘플 예

```
module "network" {  
    source = "./modules/network"  
    project_name = "shopping"  
    environment = "dev"  
    vpc_cidr     = "10.0.0.0/16"  
}  
  
module "compute" {  
    source = "./modules/compute"  
    subnet_id      = module.network.public_subnet_id  
    security_group_id = module.security.security_group_id  
}
```


15.2 Root Module과 Child Module

Child Module 역할

- 특정 목적의 인프라 구현을 담당함.
- Module을 사용하면 각 디렉터리가 하나의 명확한 책임을 갖게 됨

샘플 예

```
module "network" {
  source = "./modules/network"
  project_name = "shopping"
  environment = "dev"
  vpc_cidr    = "10.0.0.0/16"
}
```

network 모듈 책임

VPC 생성

Subnet 생성

IGW 생성

Route Table 생성

필요한 Output 제공

```
module "compute" {
  source = "./modules/compute"
  subnet_id      = module.network.public_subnet_id
  security_group_id = module.security.security_group_id
}
```

compute 모듈 책임

EC2 생성

IAM Instance Profile 연결

User Data 설정

EC2 관련 Output 제공

15.3 Module을 사용하는 이유

1. 코드 재사용

- Module을 한 번 만들면 여러 환경에서 반복 사용할 수 있음.

샘플 예

```
module "dev_vpc" {  
    source = "./modules/vpc"  
    project_name = "shopping"  
    environment = "dev"  
    vpc_cidr    = "10.0.0.0/16"  
}  
  
module "prod_vpc" {  
    source = "./modules/vpc"  
    project_name = "shopping"  
    environment = "prod"  
    vpc_cidr    = "10.1.0.0/16"  
}
```

15.3 Module을 사용하는 이유

2. 관심사 분리

- 전체 프로젝트를 역할별로 분리할 수 있음.
- 각 Module이 하나의 책임만 담당하도록 구성하면 코드 이해와 유지보수가 쉬워짐.

샘플 예

network Module

→ 네트워크 구성

security Module

→ Security Group 구성

compute Module

→ EC2 구성

database Module

→ RDS 구성

load-balancer Module

→ ALB 구성

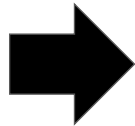
15.3 Module을 사용하는 이유

3. 코드 가독성 향상

- Module 없이 Root Module에 모든 Resource를 작성하면 복잡해짐.

샘플 예

```
aws_vpc
aws_subnet
aws_route_table
aws_route
aws_internet_gateway
aws_security_group
aws_instance
aws_lb
aws_lb_target_group
aws_lb_listener
aws_db_instance
```



```
module "network" {
  source = "./modules/network"
}

module "security" {
  source = "./modules/security"
}

module "compute" {
  source = "./modules/compute"
}

module "alb" {
  source = "./modules/alb"
}
```

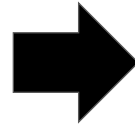
15.3 Module을 사용하는 이유

4. 유지보수성 향상

- Module 을 사용하면 특정 Module 한 곳만 수정하면 모두 적용됨.

샘플 예

```
tags = {  
  ManagedBy = "Terraform"  
  Team      = "Cloud"  
}
```



modules/ec2/main.tf 수정



Module을 사용하는 모든 환경에 동일한
정책 적용

15.3 Module을 사용하는 이유

5. 협업 효율 향상

- Module 단위로 팀원의 작업을 나눌 수 있음.
- Module의 입력과 출력 규격만 합의하면 각 팀원이 독립적으로 작업할 수 있음.

샘플 예

담당자	작업
팀원 A	Network Module
팀원 B	Security Group Module
팀원 C	EC2 Module
팀원 D	RDS Module

Network Module Output

```
vpc_id
public_subnet_ids
private_subnet_ids
```

15.4 파일 분리와 Module 분리 차이점

1. 파일만 분리한 경우

terraform-project/

├─ vpc.tf

├─ ec2.tf

└─ rds.tf

특징

- 하나의 Root Module
- 하나의 State
- 모든 Resource가 같은 Scope
- 코드 재사용 어려움

15.4 파일 분리와 Module 분리 차이점

2. Module로 분리한 경우

terraform-project/

├─ main.tf

└─ modules/

├─ vpc/

| └─ main.tf

| └─ variables.tf

| └─ outputs.tf

├─ ec2/

| └─ main.tf

| └─ variables.tf

| └─ outputs.tf

└─ rds/

├─ main.tf

├─ variables.tf

└─ outputs.tf

특징

- 각 디렉터리가 독립된 Module
- 입력과 출력 인터페이스 존재
- 다른 프로젝트에서 재사용 가능
- 내부 구현을 숨길 수 있음

수고하셨습니다.